Customer Churn Prediction & Retention Strategy

Technical Report — Final Report (Milestones 1 & 2)

Live Deployment:

customer-churn-prediction-m2-sem4.streamlit.app

Authors	Pushpendra S. Parihar, Rohan Singh & Saksham Sharma
GitHub	makeprodigy/customer-churn-prediction
Course	Generative AI — Final Project (Milestones 1 & 2)
Submitted	19 April 2026
Tech Stack	Python 3 · Scikit-Learn · LangGraph · LangChain · Gemini 2.5 Flash · Streamlit

Contents

1 Problem Statement **3**
- 1.1 Challenge Definition . . . 3
- 1.2 Business Impact . . . 3

2 Data Description **3**
- 2.1 Dataset Source . . . 3
- 2.2 Dataset Statistics . . . 4
- 2.3 Feature Inventory . . . 4
- 2.4 Class Imbalance Handling . . . 4

3 Exploratory Data Analysis (EDA) **5**
- 3.1 Numerical Feature Statistics . . . 5
- 3.2 Churn by Key Categorical Features . . . 5
- 3.3 Numerical Features: Churn vs. Non-Churn Comparison . . . 6
- 3.4 Correlation with Churn . . . 6
- 3.5 Key EDA Takeaways . . . 6

4 Methodology **6**
- 4.1 System Architecture . . . 6
- 4.2 Preprocessing Pipeline . . . 7
  - 4.2.1 Design Decisions . . . 7
- 4.3 Feature Engineering . . . 7
- 4.4 Classifier Selection & Rationale . . . 9

5 Evaluation **9**
- 5.1 Metrics Used . . . 9
- 5.2 Holdout Test Results (threshold = 0.50) . . . 10
- 5.3 Threshold Tuning . . . 10
- 5.4 Result Interpretation . . . 10

6 Optimisation **11**
- 6.1 Hyperparameter Tuning Strategy . . . 11
- 6.2 Search Spaces & Best Parameters . . . 11
- 6.3 Optimisation Impact & Design Rationale . . . 11

7 Team Contribution **12**
- 7.1 Contributor Overview . . . 12
- 7.2 Individual Responsibility Breakdown . . . 13
- 7.3 Effort Distribution Summary . . . 14

8 Milestone 2: Agentic AI Retention Engine **14**
- 8.1 Overview . . . 14
- 8.2 System Architecture . . . 14
- 8.3 Streamlit Integration . . . 14
- 8.4 Batch Collective Retention Strategy . . . 15

Conclusion **16**

References **17**

1 Problem Statement

Customer churn — the voluntary discontinuation of service by a subscriber — is one of the most economically damaging phenomena in the telecommunications industry. Acquiring a new customer is estimated to cost five to seven times more than retaining an existing one, and industry research suggests that even a **5% reduction in churn can increase profitability by 25–95%**.

1.1 Challenge Definition

Given a rich set of 19 customer attributes spanning demographics, service subscriptions, account details, and billing behaviour, the challenge is to build a reliable binary classification system that answers:

> ***"Will this customer churn within the next billing cycle?"***

More specifically, the system must:

1. **Predict** whether a given customer will churn (`Churn = Yes`) or remain (`Churn = No`).
2. **Quantify** the probability of churn to enable risk-tiered retention prioritisation.
3. **Operate** in two modes: real-time single-customer inference and bulk batch processing.
4. **Generalise** robustly to unseen customer profiles without overfitting to training noise.

1.2 Business Impact

Accurate churn prediction allows the business team to:

- Target high-risk customers with personalised retention offers *before* they leave.
- Prioritise customer success resources efficiently.
- Identify systemic issues driving disengagement (e.g., contract type, internet service quality).

2 Data Description

2.1 Dataset Source

The project uses the **IBM Telco Customer Churn** dataset, a widely-used public benchmark for binary churn classification. The dataset represents customers of a fictional telecom provider and is loaded directly from `customer_churn_datasest.csv`.

2.2 Dataset Statistics

Table 1: High-Level Dataset Summary

Property	Value
Total records	7,043 customers
Total features	20 (1 ID + 19 predictors)
Target variable	`Churn` (binary: Yes / No)
Churn rate	26.54% (1,869 churned / 5,174 retained)
Missing values	11 rows in `TotalCharges` (blank strings → NaN)

2.3 Feature Inventory

Table 2: Feature Descriptions

Feature	Type	Description
`customerID`	Categorical	Unique identifier (dropped before modelling)
`gender`	Categorical	Male / Female
`SeniorCitizen`	Binary	1 if customer is 65+, else 0
`Partner`	Binary	Has a partner (Yes/No)
`Dependents`	Binary	Has dependents (Yes/No)
`tenure`	Numerical	Months with the company (0–72)
`PhoneService`	Binary	Has phone service
`MultipleLines`	Categorical	No / Yes / No phone service
`InternetService`	Categorical	DSL / Fiber optic / No
`OnlineSecurity`	Categorical	No / Yes / No internet service
`OnlineBackup`	Categorical	No / Yes / No internet service
`DeviceProtection`	Categorical	No / Yes / No internet service
`TechSupport`	Categorical	No / Yes / No internet service
`StreamingTV`	Categorical	No / Yes / No internet service
`StreamingMovies`	Categorical	No / Yes / No internet service
`Contract`	Categorical	Month-to-month / One year / Two year
`PaperlessBilling`	Binary	Enrolled in paperless billing
`PaymentMethod`	Categorical	Electronic check / Mailed check / etc.
`MonthlyCharges`	Numerical	Monthly bill amount ($18.25–$118.75)
`TotalCharges`	Numerical	Cumulative spend (NaN-imputed)
`Churn`	**Target**	Yes (1) or No (0)

2.4 Class Imbalance Handling

The dataset exhibits moderate class imbalance with a 73.5% / 26.5% split. Three complementary strategies were applied:

1. `class_weight='balanced'` on applicable models (Logistic Regression, Decision Tree, Random Forest): automatically adjusts per-class loss penalties inversely proportional to class frequencies.

2. **SMOTE** (Synthetic Minority Oversampling Technique) via `imblearn.Pipeline`: generates synthetic churn examples inside each CV training fold only, preventing test-set contamination.
3. **ROC-AUC** as the primary CV scoring metric: unlike accuracy or F1, ROC-AUC is threshold-agnostic and correctly handles class imbalance.

3 Exploratory Data Analysis (EDA)

3.1 Numerical Feature Statistics

Table 3: Descriptive Statistics for Numerical Features

Feature	Mean	Std	Min	25%	Median	Max
`tenure`	32.37	24.56	0	9	29	72
`MonthlyCharges`	$64.76	$30.09	$18.25	$35.50	$70.35	$118.75
`TotalCharges`	$2,283	$2,267	$18.80	$401	$1,397	$8,685

3.2 Churn by Key Categorical Features

Table 4: Churn Rate by Key Categorical Variables

Feature	Value	Churn %	Insight
Contract	Month-to-month	**42.7%**	Highest single driver
	One year	11.3%	Moderate retention
	Two year	**2.8%**	Strongest retention
InternetService	Fiber optic	**41.9%**	High churn service tier
	DSL	19.0%	Lower churn
	No internet	**7.4%**	Very loyal
PaymentMethod	Electronic check	**45.3%**	Disengaged payment
	Mailed check	19.1%	
	Bank transfer (automatic)	16.7%	
	Credit card (automatic)	**15.2%**	Most loyal group
SeniorCitizen	1 (Senior)	**41.7%**	Vulnerable segment
	0 (Non-Senior)	23.6%	
PaperlessBilling	Yes	33.6%	Higher churn
	No	**16.3%**	
Dependents	No	31.3%	No dependents = more mobile
	Yes	**15.5%**	Families are stickier

3.3 Numerical Features: Churn vs. Non-Churn Comparison

Table 5: Mean Values by Churn Status

Feature	Churned	Retained	Interpretation
`tenure`	17.98 mo	37.57 mo	Churned customers leave 2× earlier
`MonthlyCharges`	\$74.44	\$61.27	Churners pay more per month
`TotalCharges`	\$1,531	\$2,555	Less lifetime value (short tenure)

3.4 Correlation with Churn

Pearson correlations between numerical features and the binary churn label:

Table 6: Pearson Correlation with `Churn`

Feature	Correlation	Direction
`tenure`	−0.352	Longer tenure ⇒ less churn
`TotalCharges`	−0.199	Higher total spend ⇒ less churn (proxy for tenure)
`MonthlyCharges`	+0.193	Higher monthly bill ⇒ more churn
`SeniorCitizen`	+0.151	Being a senior ⇒ more churn

3.5 Key EDA Takeaways

EDA Summary — Top Churn Drivers

1. **Contract type** is the dominant predictor: month-to-month customers churn at 15× the rate of two-year contract holders.
2. **Fiber optic internet** paradoxically shows high churn (41.9%), suggesting a cost/expectation mismatch.
3. **Electronic check** users are the most likely to churn (45.3%) — a disengaged payment method signals disengaged customers.
4. **Tenure** is the strongest negative predictor ($r = -0.352$): the first 12 months are critical for retention.
5. **Senior citizens** without partners/dependents represent the most at-risk demographic segment.

4 Methodology

4.1 System Architecture

The system is structured as a two-component pipeline:

1. **Offline Training** (`src/preprocessing/preprocess.py` + `src/evaluation/train.py`): Builds and serialises scikit-learn `Pipeline` objects to `.joblib` files.
2. **Online Inference** (`app.py`): Loads `.joblib` files and serves predictions via a Streamlit web application with two modes: single-customer and batch CSV.

4.2 Preprocessing Pipeline

To ensure a **leak-proof** preprocessing strategy (i.e., fitting scalers only on training data), all transformations are encapsulated inside a `ColumnTransformer` that becomes part of each model's `Pipeline`:

```
numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler())
])

categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot',  OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_transformer, numeric_features),    # tenure
       , MonthlyCharges, TotalCharges
    ('cat', categorical_transformer, categorical_features)   #
       16 categorical columns
])
```

Listing 1: Preprocessing Pipeline (src/preprocessing/preprocess.py)

4.2.1 Design Decisions

- **Median imputation** for `TotalCharges`: robust to the skewed distribution caused by early-churning customers.
- **StandardScaler** for numerical features: makes regularised models (LR, MLP) converge faster and improves coefficient interpretability.
- **One-Hot Encoding with** `handle_unknown='ignore'`: prevents errors when serving predictions on new customers with unseen category values.
- **Train/test split**: 80% / 20% stratified by random seed 42 (5,634 train / 1,409 test).

4.3 Feature Engineering

Five domain-driven features were derived from the raw columns and added to the numeric pipeline:

Table 7: Engineered Features (added in src/preprocessing/preprocess.py)

Feature	**Formula**	**Business Rationale**
`avg_monthly_charge`	TotalCharges / (tenure+1)	Lifetime average spend — distinguishes long-tenure low-spenders from new high-spenders.
`service_count`	$\sum$(add-ons = "Yes")	Higher switching cost; customers with many services churn less.
`charges_per_service`	MonthlyCharges / (service_count+1)	Proxy for value-per-service; very high values may signal overbilling risk.
`is_new_customer`	1 if tenure $\leq$ 12	New customers churn at disproportionately high rates in year one.
`is_long_term`	1 if tenure $\geq$ 48	Long-tenure customers are significantly more loyal.

4.4 Classifier Selection & Rationale

Table 8: Model Selection Rationale

Model	Library Class	Rationale
Logistic Regression	`LogisticRegression`	Gold-standard interpretable baseline. Coefficients directly indicate feature influence on churn probability. Supports `class_weight='balanced'` natively.
Decision Tree	`DecisionTreeClassifier`	Captures non-linear interactions; produces human-readable rules. Pruning via `max_depth` prevents overfitting.
MLP (Neural Net)	`MLPClassifier`	Deep-learning baseline. Learns complex feature interactions automatically; SMOTE compensates for absence of `class_weight` support.
Random Forest	`RandomForestClassifier`	Bootstrap-aggregated tree ensemble. Reduces variance, handles interactions, achieves highest F1 (62.05%).
Gradient Boosting	`GradientBoostingClassifier`	Sequential residual correction. Achieves highest CV ROC-AUC (84.59%) and highest accuracy (76.72%).

All classifiers are wrapped in an `imblearn.Pipeline`:

$$\texttt{ImbPipeline} = [\,\underbrace{\texttt{ColumnTransformer}}_{\text{Preprocessing}} \rightarrow \underbrace{\texttt{SMOTE}}_{\text{Oversampling}} \rightarrow \underbrace{\texttt{Classifier}}_{\text{Model}}\,]$$

SMOTE fires only during `.fit()` inside each CV fold and is bypassed at inference time, preventing data leakage.

5 Evaluation

5.1 Metrics Used

Given the class imbalance (26.5% positive class), multiple complementary metrics were used:

- **ROC-AUC**: Primary CV optimisation target. Threshold-agnostic; measures ranking ability across all thresholds. Unaffected by class imbalance.
- **Recall**: Of all actual churners, how many were caught? The most business-critical metric.

- **F1 Score**: Harmonic mean of precision and recall; reported for comparison.
- **Accuracy**: Overall fraction correct. Reported for completeness but not relied upon due to imbalance.

5.2 Holdout Test Results (threshold = 0.50)

All five models evaluated on 1,409 held-out customers (stratified 20% split):

Table 9: Model Evaluation Results — Test Set

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	74.24%	50.95%	78.88%	61.91%	**84.37%**
Decision Tree	75.30%	52.61%	70.05%	60.09%	80.35%
MLP (Neural Net)	73.46%	50.00%	65.78%	56.81%	79.39%
Random Forest	76.65%	54.56%	71.93%	**62.05%**	83.88%
Gradient Boosting	**76.72%**	**54.94%**	68.45%	60.95%	84.18%

5.3 Threshold Tuning

Lowering the decision threshold to 0.35 trades precision for recall, catching significantly more churners at the cost of more false alarms — generally the correct trade-off when the cost of a missed churner exceeds the cost of an unnecessary offer.

Table 10: Recall and F1 at Two Decision Thresholds

Model	Recall @0.50	Recall @0.35	F1 @0.50	F1 @0.35
Logistic Regression	78.88%	**89.57%**	61.91%	60.04%
Decision Tree	70.05%	81.28%	60.09%	58.02%
MLP	65.78%	74.60%	56.81%	57.94%
Random Forest	71.93%	84.49%	62.05%	61.30%
Gradient Boosting	68.45%	82.35%	60.95%	**61.72%**

5.4 Result Interpretation

Best Models by Criterion

- **Recall** (catching churners): Logistic Regression — 78.88% at 0.50, 89.57% at 0.35.
- **F1**: Random Forest — 62.05% at threshold 0.50.
- **ROC-AUC**: Logistic Regression 84.37%, Gradient Boosting 84.18%.
- **Business deployment**: LR at threshold 0.35 catches 9 out of 10 churners.

The reduction in raw accuracy vs. the Milestone 1 baseline (from ∼82% to ∼74–77%) is expected. Earlier models were majority-biased; SMOTE and `class_weight` correct this, trading majority-class accuracy for minority-class recall — the operationally correct objective.

6 Optimisation

6.1 Hyperparameter Tuning Strategy

All five models were optimised using **RandomizedSearchCV with 5-fold cross-validation**, scoring on ROC-AUC to correctly handle the imbalanced nature of the dataset. For algorithms lacking native `class_weight` support, SMOTE inside an imblearn `Pipeline` ensures proper minority-class attention:

```
search = RandomizedSearchCV(
    estimator=pipeline,                 # Preprocessor -> SMOTE
        -> Classifier
    param_distributions=config['params'],
    scoring='roc_auc',                  # Optimise for ROC-AUC
    cv=5,                               # 5-fold CV
    n_iter=config['n_iter'],            # Random trials per
        model
    n_jobs=-1,
    random_state=42
)
search.fit(X_train, y_train)
best_pipeline = search.best_estimator_
```

Listing 2: RandomizedSearchCV Setup (src/evaluation/train.py)

6.2 Search Spaces & Best Parameters

Table 11: RandomizedSearchCV Optimal Results

Model	Best Parameters	CV ROC-AUC
Logistic Regression	C=1.0, solver=lbfgs, class_weight=balanced	**84.73%**
Decision Tree	max_depth=5, min_samples_leaf=5, class_weight=None	81.60%
MLP (Neural Net)	hidden_layer_sizes=(50,), alpha=0.01, lr_init=0.001	79.61%
Random Forest	n_estimators=300, max_depth=10, min_samples_leaf=10	84.47%
Gradient Boosting	n_estimators=200, learning_rate=0.03, max_depth=3	84.59%

6.3 Optimisation Impact & Design Rationale

- **Logistic Regression**: Achieved the best CV ROC-AUC overall. Setting `class_weight=balanced` was sufficient to handle imbalance natively.
- **Decision Tree & Ensembles**: Constraining tree depth (`max_depth=5` for DT, `10` for RF, `3` for GB) mitigated overfitting and improved validation scores. Random Forest and Gradient Boosting yielded the highest accuracy on the test set.
- **MLP**: The smaller `(50,)` hidden layer converged better, while using SMOTE correctly handled the class imbalance.

- **Cross-validation**: 5-fold CV using ROC-AUC ensures selection of models that separate the classes effectively regardless of threshold. Subsequent tuning of the threshold to 0.35 significantly improves recall for real-world application.

7 Team Contribution

7.1 Contributor Overview

This project was developed collaboratively by three contributors. Git commit responsibilities are split in a **5:3:2 ratio** (Pushpendra 50%, Rohan 30%, Saksham 20%), with additional deliverables assigned as described below.

Table 12: Contributor Summary

Contributor	GitHub Handle	Git Share	Key Responsibilities
Pushpendra S. Parihar	`makeprodigy`	50%	ML pipeline, Agentic AI Retention Engine (LangGraph + RAG), Streamlit app, Model Training, Hyperparameter Opt., Cloud Deployment, Technical Report Review
Rohan Singh	`r0hansng`	30%	**Project demo video**, Data Sourcing, EDA, User Testing, Streamlit App collaboration
Saksham Sharma	`Saksham-9898`	20%	**Technical Report (Lead Author)**, Documentation, Testing & Validation

7.2 Individual Responsibility Breakdown

Table 13: Role-Based Contribution Split

Task Area	Pushpendra S. Parihar	Rohan Singh	Saksham Sharma
Project Setup & Scaffold	Initialised repo, gitignore, requirements	Reviewed setup	Reviewed setup
Data Pipeline	Authored `preprocess.py`: imputation, scaling, one-hot encoding	Sourced dataset, validated columns & missing values	Cross-checked encoding logic
Model Training	Authored `train.py`: LR, DT, MLP, RF, GB with RandomizedSearchCV	Reviewed model configs, analysed metrics	Validated evaluation loop outputs
Agentic AI Engine (RAG)	Architected LangGraph & RAG workflows for single/batch retention	Testing & feedback	Analysed retention metrics
Streamlit Application	Built full dashboard (`app.py`): single & batch modes, charts, session state	User testing & feedback across UI iterations, assisted in integration	Functional testing & edge-case validation
Cloud Deployment	Deployed to Streamlit Cloud; handled Secrets management	Verified live URL & behaviour	—
Hyperparameter Opt.	Implemented RandomizedSearchCV; regenerated models	Compared before/after metrics	Documented tuning results
Documentation (README)	Structured core README documentation	Reviewed content	Provided feedback & proofreading
Project Demo Video	Video script coordination	**Produced, recorded & edited full demo video**	—
Technical Report	**Content review & technical validation**	—	**LaTeX authoring, structure, and comprehensive write-up**

7.3 Effort Distribution Summary

The demo video is available at: Google Drive (Demo Video).

Table 14: Overall Contribution Split (5:3:2 Ratio)

Contributor	Git Share	%	Extra Deliverables
Pushpendra S. Parihar	Complete Lifecycle (Lead Developer)	50%	Core Architecture & AI Imple tations
Rohan Singh	EDA, Testing, UI Assistance	30%	Project demo video (link)
Saksham Sharma	Testing, Documentation	20%	Technical report (LaTeX lea thor)
Total		**100%**	

8 Milestone 2: Agentic AI Retention Engine

8.1 Overview

Milestone 2 extends the prediction system into an **agentic AI retention strategist**. When a high-risk customer is flagged ($P(\text{churn}) > 0.50$), a LangGraph workflow retrieves the most relevant retention policies from a FAISS document store and generates a personalised, policy-grounded strategy using Google Gemini 2.5 Flash.

8.2 System Architecture

1. **RAG Retrieval Node** (`src/retention/vector_store.py`): Converts the customer profile into a natural-language query, embeds it with Gemini Embedding-001, and performs a FAISS cosine similarity search against `telecom_retention_policies.md` to retrieve the top-2 relevant policy chunks.
2. **Strategy Generation Node** (`src/retention/graph_engine.py`): Sends the customer profile, churn probability, and retrieved policy context to Gemini 2.5 Flash. The model returns a bullet-pointed, policy-cited, personalised retention plan.

The two nodes are connected in a `StateGraph` (LangGraph), compiled into a callable agent:

$$\text{START} \rightarrow \underbrace{\texttt{retrieve_policies}}_{\text{RAG node}} \rightarrow \underbrace{\texttt{generate_strategy}}_{\text{LLM node}} \rightarrow \text{END}$$

8.3 Streamlit Integration

The retention engine is surfaced in two modes within the Streamlit dashboard:

1. **Single Prediction** (`ui/single_prediction.py`): After a high-risk prediction is confirmed by the ensemble, a "Generate Retention Strategy (AI)" button appears. Clicking it invokes `run_retention_engine()` and renders the Gemini-generated strategy inline with Markdown formatting.

2. **Batch Prediction** (`ui/batch_prediction.py`): After running predictions on an uploaded CSV, a "Generate Collective Retention Strategy (AI)" button appears for any batch containing predicted churners (Section 8.4).

API keys are retrieved securely from Streamlit Secrets on Streamlit Cloud, with a local `.env` fallback for development.

8.4 Batch Collective Retention Strategy

For bulk inference use cases it is impractical and expensive to generate an individual Gemini API call per churner — a dataset with 7,043 customers and 26.5% churn rate would require ≈ 1,866 sequential LLM requests. Instead, the system follows an **aggregate-then-generate** pattern:

1. **Segment Profiling** (`graph_engine._build_segment_profile()`): After batch predictions complete, all rows predicted to churn are isolated. Fifteen aggregate statistics are computed across the at-risk population:
   - Numerical: average churn probability, average tenure, average monthly charges, average active service count.
   - Categorical (mode): dominant contract type, internet service tier, payment method.
   - Proportional: % Month-to-month, % Fiber optic, % Electronic check, % Senior Citizen, % No Partner, % Paperless Billing.
2. **RAG Retrieval** (`_retrieve_batch_policies()`): The segment profile is serialised to a natural-language string and used to retrieve the top-3 most relevant policy chunks from the FAISS index — one extra chunk compared to the single-customer case to capture broader policy coverage.
3. **Strategy Generation** (`_generate_batch_strategy()`): A single `Gemini 2.5 Flash` call receives the aggregate profile and retrieved policies. The prompt instructs the model to produce a structured retention programme covering:

$$\text{START} \rightarrow \underbrace{\texttt{_retrieve_batch_policies}}_{\text{RAG node}} \rightarrow \underbrace{\texttt{_generate_batch_strategy}}_{\text{LLM node}} \rightarrow \text{END}$$

Batch Strategy Output Structure

1. **Root Cause Analysis** — why this segment is leaving.
2. **Recommended Retention Initiatives** — exact offers, discounts, contract upgrades, citing specific company policies.
3. **Estimated Impact** — projected retention lift if initiatives are executed.
4. **Sub-Segment Targeting** — differentiated treatment for demographic sub-groups (e.g. senior citizens, high-bill customers, no-partner customers).

Cost efficiency: regardless of batch size, exactly **one Gemini API call** is made. The approach shifts the analytical burden from the LLM to a deterministic aggregation function, preserving statistical representativeness while eliminating per-record API overhead.

Conclusion

This report documents the complete end-to-end development of the Customer Churn Prediction & Retention Strategy system across both milestones. Key outcomes:

- A **leak-proof imblearn Pipeline** combining feature engineering, SMOTE, and `class_weight` handling ensures correct treatment of class imbalance.
- **Five classifiers** tuned via RandomizedSearchCV (ROC-AUC): Logistic Regression (best Recall: 78.88%, 89.57% at threshold 0.35), Random Forest (best F1: 62.05%), and Gradient Boosting (best CV ROC-AUC: 84.59%).
- Churn-class **Recall improved by +13–23 percentage points** over the Milestone 1 baseline through SMOTE, class weighting, and feature engineering.
- A **LangGraph + RAG retention engine** (Gemini 2.5 Flash + FAISS) generates personalised, policy-grounded retention plans for individually flagged high-risk customers.
- A **Batch Collective Retention Strategy** aggregates all predicted churners into a single segment profile and generates one cost-effective, population-level retention programme via a single Gemini API call — efficient regardless of batch size.
- A **fully integrated Streamlit dashboard** provides single-customer inference with ensemble verdict and AI strategy, bulk batch processing with collective retention programme, and a metrics comparison page.
- The system is **deployment-ready** on Streamlit Community Cloud with secure API key handling via Streamlit Secrets.

References

1. IBM. (2019). *Telco Customer Churn Dataset*. IBM Sample Data Sets. https://www.kaggle.com/datasets/blastchar/telco-customer-churn

2. Pedregosa, F., Varoquaux, G., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://scikit-learn.org

3. Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets. *JMLR*, 18(17), 1–5. https://imbalanced-learn.org

4. Chase, H. (2022). *LangChain*. GitHub. https://github.com/langchain-ai/langchain

5. LangChain AI. (2024). *LangGraph — Build stateful, multi-actor applications with LLMs*. https://langchain-ai.github.io/langgraph/

6. Google DeepMind. (2024). *Gemini 2.5 Flash — Technical Overview*. https://ai.google.dev/gemini-api/docs

7. Streamlit Inc. (2023). *Streamlit — The fastest way to build and share data apps*. https://streamlit.io

8. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

9. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56, 51–56. https://pandas.pydata.org

10. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95. https://matplotlib.org